

Criptografia de Chave Pública — do zero, bem básico

Guia de estudo pros vídeos V6 (chave pública), V7 (RSA), V8 (curvas elípticas)

Guia de estudo pros vídeos V6 (chave pública), V7 (RSA), V8 (curvas elípticas). Linguagem simples. Exemplos com números pequenos. Lê de cima pra baixo.

0. A ideia central (leia isso primeiro)

Cripto simétrica: mesma chave pra fechar e abrir. Problema: como as duas partes combinam a chave sem se encontrar?

Chave pública resolve isso. Cada pessoa tem **duas** chaves:

- **pública** — todo mundo pode ver
- **privada** — só o dono

Regras:

Quero...	Uso
Mandar segredo pra Alice	cifro com a pública da Alice , ela abre com a privada dela
Provar que fui eu que escrevi	assino com a minha privada , todos verificam com a minha pública

Tudo isso funciona porque existe uma operação **fácil de fazer, difícil de desfazer**:

- multiplicar dois primos é fácil; fatorar o produto é difícil → **RSA**
- calcular $g^x \bmod p$ é fácil; achar x a partir do resultado é difícil → **Diffie-Hellman, ElGamal, DSA**
- somar pontos numa curva elíptica é fácil; achar quantas vezes somou é difícil → **ECC, ECDH, ECDSA**

Isso é “função de mão única” (one-way function). **É a base de tudo.**

1. Fundamentos de Teoria dos Números

1.1 Divisibilidade e primos

- **$a \mid b$ (“a divide b”)** — existe inteiro k com $b = a \cdot k$.
 - *Exemplo:* $3 \mid 12$ porque $12 = 3 \cdot 4$.

- **Número primo** — inteiro > 1 divisível só por 1 e por ele mesmo.
 - Exemplos: 2, 3, 5, 7, 11, 13, ...
- **Teorema fundamental da aritmética** — todo inteiro > 1 se escreve de **um único jeito** como produto de primos.
 - Exemplo: $360 = 2^3 \cdot 3^2 \cdot 5$.
- **mdc / gcd** — maior número que divide os dois ao mesmo tempo.
 - Exemplo: $\gcd(12, 18) = 6$.
 - Se $\gcd(a, b) = 1 \rightarrow$ são **coprímos** (primos entre si).

1.2 Aritmética modular

- **$a \bmod n$** — resto da divisão de a por n .
 - Exemplo: $17 \bmod 5 = 2$.
- **$a \equiv b \pmod{n}$ (“congruente”)** — a e b deixam o mesmo resto ao dividir por n .
 - Exemplo: $17 \equiv 2 \pmod{5}$.
- **Intuição do relógio** — aritmética modular é como um relógio que dá a volta.
 - Exemplo: $\bmod 12$, 13 horas = 1 hora.
- **O que continua valendo:**
 - soma, subtração e multiplicação funcionam normalmente $\rightarrow (a+b) \bmod n, (a \cdot b) \bmod n$
 - **divisão NÃO existe direto** $\rightarrow$ em vez de dividir, multiplica-se pelo **inverso**
- **Inverso multiplicativo** — a^{-1} é o número tal que $a \cdot a^{-1} \equiv 1 \pmod{n}$.
 - Exemplo: $\bmod 7, 3 \cdot 5 = 15 \equiv 1 \rightarrow$ inverso de 3 é 5.
 - **Regra de ouro — decora:** existe inverso de $a \bmod n \iff \gcd(a, n) = 1$.

1.3 Grupo — a estrutura que une tudo

- **Definição resumida:** um **grupo** é um conjunto G + **uma** operação $*$ que cumpre rigorosamente 4 axiomas.
 - Pensa como “um conjunto com uma calculadora própria que nunca te joga para fora”.
- **Os 4 axiomas — um por bullet para não confundir:**
 - **1. Fechamento** — se pegares dois elementos quaisquer de G e aplicares $*$, o resultado continua dentro de G .
 - * Sem isso a operação te jogaria para fora do conjunto.
 - **2. Associatividade** — a forma de agrupar não muda o resultado: $(a*b)*c = a*(b*c)$.
 - **3. Elemento neutro (identidade) e** — existe um elemento especial que não altera ninguém: $a*e = a$.
 - * Exemplos: 0 na adição, 1 na multiplicação.
 - **4. Elemento inverso** — para cada a existe um a^{-1} que desfaz a operação: $a*a^{-1} = e$.
- **Grupo abeliano (comutativo):**
 - Se além dos 4 axiomas ainda vale $a*b = b*a$ para todo a, b , o grupo é **abeliano**.
 - Em criptografia usamos quase sempre grupos abelianos.
- **Nomenclatura — onde isso se encaixa na matemática?**
 - **Álgebra Abstrata** (ou Álgebra Moderna) é a área geral.
 - **Teoria de Grupos** é o sub-tópico exato que estuda grupos.

- **Aritmética Modular** é o nome quando aplicamos isso a restos de divisão ($\text{mod } n$).
 - * *Exemplo da apostila:* $\mathbb{Z}_7^*$ é o **Grupo Multiplicativo de Inteiros Módulo 7** — um **Grupo Cíclico Finito**.
 - *Finito* porque tem poucos elementos (6).
 - *Cíclico* porque um gerador dá a volta e gera todo mundo.
- **Hierarquia de estruturas (decora a escada):**
 - * **Grupo:** 1 operação + 4 regras.
 - * **Anel:** 2 operações (geralmente soma e multiplicação).
 - * **Corpo (Field/Campo):** anel onde todo elemento (exceto 0) tem inverso multiplicativo — onde a divisão funciona perfeitamente.
- **Por que importa para ti:**
 - Todos os esquemas abaixo (**DH, ElGamal, ECC**) são o **mesmo esquema** escrito em grupos diferentes.
 - Entende **grupo uma vez**, entende todos os protocolos.

Ordem — Grupo vs. Elemento (a parte que mais confunde)

- **Ordem do grupo $|G|$** — quantidade total de elementos que o grupo tem.
 - *Exemplo:* $|\mathbb{Z}_7^*| = 6$ porque $\mathbb{Z}_7^* = \{1, 2, 3, 4, 5, 6\}$.
- **Ordem de um elemento a** — o **menor** expoente $k > 0$ necessário para que, ao aplicar a operação a si mesmo k vezes (a^k), **volte ao elemento neutro e** .
 - *Intuição da pista circular:* imagina que estás a caminhar à volta de uma pista. A ordem é o número mínimo de passos até voltar exatamente à largada ($e = 1$ na multiplicação).
 - *É sempre “elevado a algo”?*
 - * **Em grupo multiplicativo (o da cripto, como na imagem)** $\rightarrow$ sim, é a^k .
 - * **Em grupo aditivo** (operação é $+$) $\rightarrow$ seria $k \cdot a = a + a + \dots + a = 0$ até chegar no 0.
- **Exemplo passo a passo — ordem do 2 em $\mathbb{Z}_7^*$:**
 - $2^1 = 2$
 - $2^2 = 4$
 - $2^3 = 8 \equiv 1 \pmod{7} \rightarrow$ **chegou no neutro!**
 - Logo, **ordem do 2 = 3**. Ele forma um ciclo curto que se repete: $2 \rightarrow 4 \rightarrow 1 \rightarrow 2 \rightarrow 4 \rightarrow 1 \dots$
- **Teorema de Lagrange — desmistificado:**
 - **Enunciado:** a ordem de **qualquer** elemento **divide** exatamente a ordem total do grupo.
 - **Teste prático com $|G| = 6$:**
 - * ordem do 2 = 3 $\rightarrow$ 3 divide 6? **Sim** ($6 \div 3 = 2$, resto zero) ✓
 - * ordem do 3 = 6 $\rightarrow$ 6 divide 6? **Sim** ✓
 - * **Consequência:** nunca poderá existir um elemento de ordem 4 num grupo de ordem 6, porque 4 não divide 6.
 - **Consequência direta (fórmula que aparece na imagem): $a^{|G|} = e$ sempre, para qualquer a :**
 - * *Por quê?* Se $\text{ord}(a)$ divide $|G|$, então $|G| = k \cdot \text{ord}(a)$ e $a^{|G|} = (a^{\text{ord}(a)})^k = e^k = e$.
 - * *Teste sem fazer conta longa:*
 - $2^6 = (2^3)^2 = 1^2 = 1$ ✓
 - $3^6 = 1$ ✓

Grupo cíclico e gerador (elemento primitivo)

- **Definição:**

- Grupo é **cíclico** se existe **pelo menos um** elemento especial **g** — o **gerador** — capaz de gerar **todos** os outros só sendo elevado sucessivamente.
- Fórmula: $G = \{ g^1, g^2, g^3, \dots, g^{|G|} = e \}$.

- **Exemplo completo que prova que 3 é gerador de $\mathbb{Z}_7^*$:**

- $3^1 = 3$
- $3^2 = 9 \equiv 2$
- $3^3 = 6$
- $3^4 = 18 \equiv 4$
- $3^5 = 12 \equiv 5$
- $3^6 = 15 \equiv 1$ (neutro)
- $\rightarrow$ Gerou $\{1, 2, 3, 4, 5, 6\}$ **todos**. Logo, 3 é gerador. Ordem = 6 = $|G|$.

- **Contra-exemplo — por que 2 NÃO é gerador:**

- $2^1 = 2, 2^2 = 4, 2^3 = 1, 2^4 = 2 \dots \rightarrow$ só gera $\{2, 4, 1\}$ (ciclo de tamanho 3).
- Ordem 3 $\neq 6 \rightarrow$ não gerou o grupo todo.

- **Tabela completa — ordem de TODOS os elementos de $\mathbb{Z}_7^*$ (o que pediste):**

Elemento a	Potências até voltar ao 1	Ordem ord(a)	É gerador?	Comentário
1	$1^1 = 1$	1	Não	Neutro sempre tem ordem 1
2	2, 4, 1	3	Não	Ciclo curto
3	3, 2, 6, 4, 5, 1	6	Sim	Gera tudo
4	4, 2, 1	3	Não	Mesmo ciclo do 2 (é 2^2)
5	5, 4, 6, 2, 3, 1	6	Sim	Outro gerador!
6	6, 1	2	Não	$6 \equiv -1$, sempre ordem 2

- *Repara:* todas as ordens (1, 2, 3, 6) **dividem** $|G| = 6$ — **Lagrange conferido na prática!**
- *Regra prática que cai em prova:* número de geradores = $\varphi(|G|)$. Aqui $\varphi(6)=2 \rightarrow$ exatamente 3 e 5.

1.4 Os dois grupos mod N

$\mathbb{Z}_N$ (aditivo) = $\{0, 1, \dots, N-1\}$ com **soma** mod N. Sempre grupo, sempre cíclico (gerador = 1), tamanho N.

$\mathbb{Z}_N^*$ (multiplicativo) = os elementos de $\{1, \dots, N-1\}$ que são **coprimos com N**, com **multiplicação** mod N. Só entram os coprimos porque só eles têm inverso.

- $\mathbb{Z}_7^* = \{1, 2, 3, 4, 5, 6\} \rightarrow 6$ elementos (7 é primo, todos entram)
- $\mathbb{Z}_{10}^* = \{1, 3, 7, 9\} \rightarrow 4$ elementos (2, 4, 5, 6, 8 compartilham fator com 10)

$\mathbb{Z}_p^*$ com p primo é **sempre cíclico**. Isso é o que faz o Diffie-Hellman funcionar.

1.5 Função phi de Euler

$\varphi(N)$ = quantos números de 1 a N são coprimos com N = **tamanho de $\mathbb{Z}_N^*$** .

Regras que você precisa:

Caso	Fórmula	Exemplo
p primo	$\varphi(p) = p-1$	$\varphi(7)=6$
p, q primos distintos	$\varphi(pq) = (p-1)(q-1)$	$\varphi(15)=2 \cdot 4=8$
p^k	$\varphi(p^k) = p^k - p^{k-1}$	$\varphi(8)=8-4=4$

A **segunda linha é o coração do RSA**. Quem sabe p e q calcula $\varphi(N)$ na hora. Quem só tem N teria que fatorar — inviável se N é grande.

1.6 Euler e Fermat

Teorema de Euler: se $\gcd(a, N)=1$ então $a^{\varphi(N)} \equiv 1 \pmod{N}$.

Pequeno Teorema de Fermat (caso especial, N primo p): $a^{p-1} \equiv 1 \pmod{p}$ para a não múltiplo de p .

Uso prático: **reduzir expoentes**. $3^{100} \bmod 7 \rightarrow 100 \bmod 6 = 4 \rightarrow 3^4 = 81 \equiv 4 \pmod{7}$. É isso que permite RSA “desfazer” a cifragem.

1.7 Teorema Chinês do Resto (CRT) — opcional

Se n_1, n_2 coprimos, o sistema $x \equiv a_1 \pmod{n_1}$ e $x \equiv a_2 \pmod{n_2}$ tem solução única $\bmod n_1 n_2$.

Uso em RSA: em vez de calcular $\bmod N = pq$ (números enormes), calcula-se separado $\bmod p$ e $\bmod q$ (metade do tamanho) e recombina. **~4x mais rápido** na decifragem/assinatura. Cuidado: implementação com CRT é vulnerável a ataques de falha (fault attacks) se não checar o resultado.

2. Teoria dos Números Algorítmica

Ou: “quais contas eu consigo fazer rápido”.

2.1 Euclides (gcd)

Ideia: $\gcd(a, b) = \gcd(b, a \bmod b)$, até o resto ser 0.

$\gcd(252, 198)$

$$252 = 1 \cdot 198 + 54$$

$$198 = 3 \cdot 54 + 36$$

$$54 = 1 \cdot 36 + 18$$

$$36 = 2 \cdot 18 + 0 \rightarrow \gcd = 18$$

Rápido: $\sim \log(n)$ passos.

2.2 Euclides Estendido $\rightarrow$ inverso

Além do gcd, acha x, y com $a \cdot x + b \cdot y = \gcd(a, b)$.

Se $\gcd(a, N)=1$: $a \cdot x + N \cdot y = 1 \rightarrow \text{mod } N$ vira $a \cdot x \equiv 1 \rightarrow x$ é o inverso de a .

É assim que o RSA calcula d a partir de e . Único jeito prático.

2.3 Exponenciação modular (square-and-multiply)

Calcular $a^b \text{ mod } n$ sem fazer b multiplicações.

Escreve b em binário, e para cada bit: eleva ao quadrado; se o bit é 1, multiplica por a . Reduz mod n a cada passo (nunca deixa crescer).

$3^{13} \text{ mod } 7, 13 = 1101_2$:

bit 1 $\rightarrow 1^2 \cdot 3 = 3$

bit 1 $\rightarrow 3^2 \cdot 3 = 27 \equiv 6$

bit 0 $\rightarrow 6^2 = 36 \equiv 1$

bit 1 $\rightarrow 1^2 \cdot 3 = 3 \rightarrow$ resposta 3

Custo: $\sim \log_2(b)$ passos em vez de b . **2048 passos em vez de 2^{2048}** . É o que torna RSA possível.

Versão ingênua vaza os bits do expoente por tempo/consumo $\rightarrow$ *side-channel*. Na prática usa-se versões de tempo constante (Montgomery ladder).

2.4 Teste de primalidade

Precisamos gerar primos gigantes (1024+ bits). Não dá pra fatorar pra testar — usa-se testes **probabilísticos**.

Teste de Fermat: escolhe a aleatório, testa $a^{(n-1)} \equiv 1 \pmod{n}$. Se falhar $\rightarrow$ composto, certeza. Se passar $\rightarrow$ *provavelmente* primo. Problema: **números de Carmichael** (561, 1105, ...) passam pra todo a . Fraco.

Miller–Rabin (o usado de verdade): escreve $n-1 = 2^s \cdot d$ com d ímpar. Testa se $a^d \equiv 1$ ou se algum $a^{(2^r \cdot d)} \equiv -1$. Se nenhum, é composto. - cada rodada com a aleatório: erro $\leq 1/4$ - k rodadas: erro $\leq 4^{(-k)}$. Com $k=40$ o erro é menor que falha de hardware.

Gerar primo de n bits: 1. sorteia n bits aleatórios, força bit mais alto e bit mais baixo = 1 (tamanho certo + ímpar) 2. divisão de teste por primos pequenos (elimina $\sim 80\%$ rápido) 3. Miller–Rabin k vezes 4. falhou $\rightarrow$ volta ao passo 1

Pelo Teorema dos Números Primos, ~ 1 em cada $\ln(2^n)$ números é primo $\rightarrow$ pra 1024 bits, ~ 1 em 710. Poucas tentativas.

2.5 Ataques — fatoração

Algoritmo	Quando funciona	Custo
Divisão de teste	fatores pequenos	$O(\sqrt{N})$
Pollard $p-1$	se $p-1$ só tem fatores pequenos (“smooth”)	rápido nesse caso
Pollard rho	genérico, fatores médios	$O(N^{(1/4)})$
Crivo quadrático (QS)	N até ~ 100 dígitos	subexponencial

Algoritmo	Quando funciona	Custo
Crivo de corpo numérico (NFS)	melhor conhecido hoje	subexponencial

→ Por isso p e q do RSA são escolhidos grandes, aleatórios, de tamanho parecido (mas não perto demais) e com $p-1$ não-smooth.

2.6 Ataques — log discreto (DL)

Achar x em $g^x = h$.

Algoritmo	Custo	Obs
Força bruta	$O(n)$	n = ordem do grupo
Baby-step giant-step	$O(\sqrt{n})$ tempo e memória	troca memória por tempo
Pollard rho	$O(\sqrt{n})$ tempo, memória ~ 0	o usado na prática
Pohlig–Hellman	quebra em cada fator primo de n	por isso a ordem tem que ter um fator primo grande
Index calculus	subexponencial	só funciona em Z^*_p, NÃO em curvas elípticas

Essa última linha é a resposta de “por que ECC usa chaves menores”.

2.7 Tamanhos de chave recomendados

Segurança	Simétrico	RSA / DH (Z^*_p)	ECC
80 bits (legado)	80	1024	160
112 bits	112	2048	224
128 bits (padrão hoje)	128	3072	256
192 bits	192	7680	384
256 bits	256	15360	512

RSA cresce muito porque o NFS é subexponencial; ECC cresce linearmente ($2\times$ a segurança) porque só existe ataque genérico $O(\sqrt{n})$.

3. Hipóteses de Dificuldade (hardness assumptions)

Nada em cripto de chave pública é *provado* seguro. Tudo se apoia em “acreditamos que esse problema é difícil”.

Hipótese da fatoração — dado $N = pq$ com p, q primos grandes aleatórios, é inviável achar p e q .

Hipótese RSA — dado N , e e $y = x^e \bmod N$, é inviável achar x . (Mais forte: se você fatora, você quebra RSA. O contrário não se sabe.)

Log discreto (DL) — em grupo cíclico G de gerador g : dado $h = g^x$, achar x é inviável.

Diffie-Hellman computacional (CDH) — dados g^a e g^b , calcular g^{ab} .

Diffie-Hellman decisional (DDH) — dados g^a , g^b , g^c , decidir se $c = ab$ ou se c é aleatório. Mais forte que CDH.

Relação: DDH difícil $\Rightarrow$ CDH difícil $\Rightarrow$ DL difícil. (quebrar DL quebra tudo; DDH é a hipótese mais exigente)

Cuidado: DDH é falsa em $\mathbb{Z}_p^*$ inteiro, porque o símbolo de Legendre vaza 1 bit. Solução: trabalhar no **subgrupo de ordem prima q** dentro de $\mathbb{Z}_p^*$ (escolhe $p = 2q+1$, “primo seguro”, e usa os resíduos quadráticos). Isso também mata Pohlig–Hellman. Por isso todo padrão fala em (p, q, g) .

Curvas elípticas — mesmo jogo, outro grupo. Como não existe index calculus lá, o melhor ataque é $O(\sqrt{n})$ genérico $\rightarrow$ chaves $\sim 12\times$ menores pra mesma segurança.

4. Troca de Chaves

4.1 O problema da distribuição de chaves

Com n pessoas e só cripto simétrica: $n(n-1)/2$ chaves. 1000 pessoas = ~ 500 mil chaves, cada uma entregue por canal seguro. Inviável. Essa é a **motivação histórica** da chave pública (Diffie–Hellman, 1976).

4.2 Diffie–Hellman (DHKE)

Público: primo p , gerador g .

Alice		Bob
a aleatório		b aleatório
$A = g^a \bmod p$	$\xrightarrow{\quad A \quad}$	
	$\xleftarrow{\quad B \quad}$	
$s = B^a = g^{ab}$		$B = g^b \bmod p$
		$s = A^b = g^{ab}$

Os dois chegam no **mesmo s** sem nunca mandá-lo. Espião vê p, g, A, B e precisaria resolver CDH.

Exemplo numérico ($p=23, g=5$): $a=6 \rightarrow A=8$; $b=15 \rightarrow B=19$; $s = 19^6 \bmod 23 = 2$ e $s = 8^{15} \bmod 23 = 2$. ✓

Na prática s passa por um **KDF** antes de virar chave de AES.

4.3 Man-in-the-middle

DH puro **não autentica ninguém**. Mallory no meio:

Alice	$\xrightarrow{\quad A \quad}$	Mallory	$\xrightarrow{\quad M \quad}$	Bob
Alice	$\xleftarrow{\quad M \quad}$	Mallory	$\xleftarrow{\quad B \quad}$	Bob

Mallory tem uma chave com Alice e outra com Bob, decifra e re-cifra tudo. Os dois acham que estão seguros.

Conserto: autenticar as mensagens do DH — assinatura digital (§6) + certificado (§6.5). É exatamente isso que o TLS faz.

5. Cifragem de Chave Pública

5.1 Sintaxe e segurança

Três algoritmos: - $\text{Gen}() \rightarrow (\text{pk}, \text{sk})$ - $\text{Enc}(\text{pk}, m) \rightarrow c$ - $\text{Dec}(\text{sk}, c) \rightarrow m$

CPA-segurança (chosen-plaintext): adversário escolhe dois textos m_0, m_1 , recebe a cifra de um deles, não consegue adivinhar qual. $\rightarrow$ **Exige cifragem aleatorizada**. Determinístico nunca é CPA-seguro (basta o atacante cifrar m_0 e comparar).

CCA-segurança (chosen-ciphertext): o adversário ainda pode pedir a decifragem de cifras que ele escolher (menos a alvo). É o que se exige na prática — o mundo real tem atacantes que mandam cifras malformadas e observam a reação. CCA implica **não-maleabilidade**: não dá pra pegar c e produzir c' que decifra em algo relacionado.

5.2 Cifragem híbrida / KEM-DEM

Chave pública é **lenta** e só cifra mensagens pequenas. Então:

1. **KEM** (Key Encapsulation) — usa chave pública pra transportar uma chave simétrica k aleatória
2. **DEM** (Data Encapsulation) — cifra a mensagem de verdade com k usando AES-GCM

$c = (\text{Enc}_{\text{pk}}(k), \text{AES}_k(m))$

Todo sistema real é híbrido. TLS, PGP, Signal, age — todos. Regra: a segurança do híbrido é a do elo mais fraco (KEM CCA + DEM CCA $\rightarrow$ híbrido CCA).

5.3 ElGamal (baseado em DDH)

É o DH transformado em cifra.

- **Chaves**: $\text{sk} = x, \text{pk} = h = g^x$
- **Cifrar m** : sorteia r , envia $c = (g^r, m \cdot h^r)$
- **Decifrar** (c_1, c_2): $m = c_2 / c_1^x$, porque $c_1^x = g^{rx} = h^r$

Propriedades: - **aleatorizado** (r novo a cada vez) $\rightarrow$ CPA-seguro sob DDH $\checkmark$ - **expande $2\times$** a mensagem - **maleável**: multiplica c_2 por t e a mensagem vira $m \cdot t \rightarrow$ **NÃO é CCA-seguro** (essa maleabilidade é homomórfica e é usada de propósito em voto eletrônico) - **r nunca** pode repetir

KEM baseado em DDH (versão moderna, é o que se usa): manda só $c_1 = g^r$ e define a chave como $k = \text{KDF}(h^r)$. Sem multiplicação, sem maleabilidade. Isso é o **ECIES** / **HPKE**.

5.4 RSA cru (“textbook”)

- **Gen**: escolhe primos $p, q; N = pq; \varphi(N) = (p-1)(q-1)$; escolhe e com $\text{gcd}(e, \varphi) = 1$ (quase sempre $e = 65537$); calcula $d = e^{-1} \bmod \varphi(N)$ (Euclides estendido). $\text{pk} = (N, e), \text{sk} = d$.
- **Enc**: $c = m^e \bmod N$
- **Dec**: $m = c^d \bmod N$

Por que volta? $c^d = m^{ed}$, e $ed \equiv 1 \pmod{\varphi} \rightarrow ed = 1 + k\varphi \rightarrow m^{(1+k\varphi)} = m \cdot (m^\varphi)^k \equiv m \cdot 1 = m$ (Euler).

Exemplo miniatura: $p=3, q=11 \rightarrow N=33, \varphi=20. e=3 \rightarrow d=7$ ($3 \cdot 7 = 21 \equiv 1 \bmod 20$). $m=4: c = 4^3 = 64 \equiv 31. 31^7 \bmod 33 = 4. \checkmark$

Por que é INSEGURO assim: 1. **determinístico** → não é CPA-seguro; dá pra montar dicionário (ex.: cifrar “sim”/“não” e comparar) 2. **maleável** → $c \cdot s^e$ decifra em $m \cdot s$ 3. **m pequeno** → se $m^e < N$ não tem redução nenhuma, basta tirar a raiz e -ésima nos inteiros 4. **mesmo m pra e destinatários** → ataque de broadcast de Håstad (via CRT)

Nunca use RSA sem padding. Nunca.

5.5 Padding — PKCS #1 v1.5

EM = 0x00 || 0x02 || PS (bytes aleatórios não-nulos) || 0x00 || m

Adiciona aleatoriedade → resolve o determinismo. Mas: **Ataque de Bleichenbacher (1998)**: se o servidor responde diferente para padding válido/inválido, o atacante decifra a mensagem com ~1 milhão de consultas (“million message attack”). Um oráculo de 1 bit basta. Ressurgiu como **ROBOT** em 2017 — ainda havia servidores vulneráveis. → v1.5 para **cifragem** é legado. Evitar.

5.6 OAEP (PKCS #1 v2)

Padding com estrutura Feistel usando duas funções de hash (máscaras):

$m \rightarrow [m \parallel \text{padding} \parallel \text{hash}(\text{label})] \oplus \text{MGF}(r) \quad \text{e} \quad r \oplus \text{MGF}(\text{daquilo})$

- CCA-seguro **no modelo do oráculo aleatório** (ROM)
- decifragem devolve erro **único** e em tempo constante — sem oráculo pra Bleichenbacher
- **é o que você deve usar** se for usar RSA pra cifrar

Resumo prático: **cifra nova hoje** → **ECDH/HPKE** ou **RSA-OAEP**. Nunca RSA cru, nunca v1.5.

6. Assinaturas Digitais

6.1 Sintaxe e segurança

- $\text{Gen}() \rightarrow (pk, sk)$
- $\text{Sign}(sk, m) \rightarrow \sigma$
- $\text{Verify}(pk, m, \sigma) \rightarrow 0/1$

Assina com a privada, verifica com a pública. É o inverso da cifragem.

Segurança = **EUFCMA** (existential unforgeability under chosen-message attack): mesmo vendo assinaturas de mensagens que ele escolheu, o atacante não consegue produzir assinatura válida para **nenhuma** mensagem nova.

Dá três coisas: **autenticidade, integridade, não-repúdio** (o MAC dá as duas primeiras, mas não a terceira — porque com MAC os dois lados têm a mesma chave).

6.2 Hash-and-sign

Assinatura pública é lenta e limitada em tamanho. Então assina-se o **hash**:

$\sigma = \text{Sign}(sk, H(m))$

Segurança depende de H ser **resistente a colisão**: se o atacante acha $m \neq m'$ com $H(m) = H(m')$, uma assinatura serve pras duas. $\rightarrow$ É por isso que MD5 e SHA-1 estão banidos (ver ataque de colisão em certificados, Flame/SHattered). Use SHA-256+.

6.3 Assinaturas RSA

RSA cru: $\sigma = m^d \bmod N$, verifica $\sigma^e \equiv m$. **Quebrado**: - forja trivial: escolhe σ qualquer, a “mensagem” é σ^e - multiplicativa: $\sigma_1 \cdot \sigma_2$ assina $m_1 \cdot m_2$

RSA-FDH (Full Domain Hash): $\sigma = H(m)^d \bmod N$, com H mapeando pra todo $\mathbb{Z}_N$. Provável EUF-CMA no ROM. Conceitualmente o certo; na prática o padrão é **RSA-PSS** (probabilístico, com salt, prova mais apertada).

PKCS #1 v1.5 para assinatura: $EM = 0x00 || 0x01 || 0xFF \dots 0xFF || 0x00 || \text{DigestInfo} || H(m)$. Padding determinístico, sem salt. Continua **onipresente** (certificados X.509, JWT RS256). Não é quebrado em si — mas verificadores relaxados sofrem o **ataque de Bleichenbacher de assinatura (e=3)**: se a implementação não checa que o padding preenche o bloco *inteiro*, dá pra forjar assinatura só tirando raiz cúbica. $\rightarrow$ verificação correta: **re-codificar e comparar byte a byte**, nunca fazer parsing.

Para assinar hoje: **Ed25519** ou **ECDSA P-256**; RSA-PSS se precisar de RSA.

6.4 DSA e ECDSA

DSA trabalha no subgrupo de ordem prima q dentro de $\mathbb{Z}_p^*$. ECDSA é o mesmo em curva elíptica. Assinatura = par (r, s) .

ECDSA — **assinar** m com privada d , gerador G de ordem n : 1. $e = H(m)$ (truncado pro tamanho de n) 2. sorteia **nonce** k em $[1, n-1]$ 3. $R = kG$; $r = R.x \bmod n$ (se 0, refaz) 4. $s = k^{-1}(e + r \cdot d) \bmod n$ (se 0, refaz) 5. assinatura = (r, s)

Verificar com pública $Q = dG$: $u_1 = e \cdot s^{-1}$, $u_2 = r \cdot s^{-1}$, $P = u_1G + u_2Q$; válido se $P.x \bmod n == r$.

! O k é a parte mortal: - k repetido em duas mensagens $\rightarrow$ duas equações, duas incógnitas $\rightarrow$ a chave privada sai por álgebra simples. Foi assim que quebraram o **PS3 da Sony (2010)** e carteiras Bitcoin com RNG ruim no Android (2013). - k com poucos bits enviesados $\rightarrow$ ataques de rede (lattice) recuperam a chave com algumas centenas de assinaturas.

Conserto: RFC 6979 — k determinístico derivado de $\text{HMAC}(sk, H(m))$. Ou use **Ed25519**, que já nasce determinístico e sem essas armadilhas.

Também: ECDSA é **maleável** — $(r, -s \bmod n)$ também é válida. Origem da maleabilidade de transações no Bitcoin; resolvido exigindo s “low”.

6.5 Certificados e PKI

Volta ao MITM do §4.3: **como sei que essa chave pública é mesmo da Alice?**

Certificado = declaração assinada por uma Autoridade Certificadora (CA): $>$ “a chave pública pk pertence a **exemplo.com**, válida até 2027” — assinado pela CA

Verificação = **cadeia de confiança**:

Raiz (auto-assinada, já no seu S0/navegador)

└─ CA intermediária

└─ certificado do site

Você confia na raiz (ela veio pré-instalada), ela avalia a intermediária, que avalia o site. Só precisa confiar em poucas raízes, não em milhões de sites.

Conteúdo de um X.509: sujeito, chave pública, emissor, validade, usos permitidos, número de série, assinatura da CA.

Revogação (chave vazou antes do vencimento): CRL (listas, grandes e lentas), **OCSP** (consulta online, vaza privacidade), **OCSP stapling** (servidor entrega a prova — melhor), validade curta (Let's Encrypt, 90 dias — a abordagem que venceu na prática).

Fraquezas reais da PKI: qualquer CA pode emitir pra qualquer domínio (DigiNotar, 2011 — CA comprometida emitiu certificado do Google); mitigação atual = **Certificate Transparency** (todo certificado vai num log público auditável).

7. Curvas Elípticas (V8)

7.1 O que é

Sobre um corpo finito F_p , a curva de Weierstrass:

$$y^2 = x^3 + ax + b \pmod{p}, \text{ com } 4a^3 + 27b^2 \neq 0 \text{ (evita pontos singulares)}$$

Os **pontos** (x, y) que satisfazem isso, **mais** um ponto especial no infinito 0 , formam um **grupo**.

7.2 A lei de grupo (operação = “soma de pontos”)

- **Identidade**: $0. P + 0 = P$
- **Inverso**: $-P = (x, -y)$ (reflexão no eixo x)
- **$P + Q$** : traça a reta por P e Q ; ela corta a curva num terceiro ponto; **reflete no eixo x** $\rightarrow$ esse é $P+Q$
- **$P + P$ (dobra)**: usa a **tangente** em P , mesma regra

Fórmulas (inclinação λ): $- P \neq Q: \lambda = (y_2 - y_1) / (x_2 - x_1)$ - $P = Q: \lambda = (3x_1^2 + a) / (2y_1)$ - e então $x_3 = \lambda^2 - x_1 - x_2$, $y_3 = \lambda(x_1 - x_3) - y_1$

(divisão = multiplicar pelo inverso mod p , §2.2)

Multiplicação escalar $kP = P+P+\dots+P$ (k vezes) — calculada por double-and-add, o análogo exato do square-and-multiply.

Dicionário: multiplicação mod $p \leftrightarrow$ soma de pontos; exponenciação $g^x \leftrightarrow xP$. **É o mesmo grupo abstrato.** Todo esquema traduz direto.

7.3 ECDLP

Dado P e $Q = kP$, achar k .

Fácil calcular kP ; inviável voltar. **Não existe index calculus para curvas elípticas** — só ataques genéricos $O(\sqrt{n})$ (Pollard rho).

Consequência: chave de 256 bits em curva $\approx$ 128 bits de segurança $\approx$ RSA de 3072 bits. Daí: chaves menores, assinaturas menores, contas mais rápidas, menos bateria.

Curvas a evitar: ordem com só fatores pequenos (Pohlig–Hellman), curvas anômalas ($\#E = p$), curvas supersingulares (ataques MOV/pareamento). → **Use curvas padronizadas:** P-256, P-384, secp256k1 (Bitcoin/Ethereum), Curve25519 (a mais amigável a implementação).

7.4 Parâmetros de domínio

(p, a, b, G, n, h) — corpo, coeficientes, ponto gerador, ordem de G (primo), e cofator $h = \#E/n$.

Chave privada: d aleatório em $[1, n-1]$. Chave pública: $Q = dG$.

7.5 ECDH

Idêntico ao DH, trocando expoente por escalar:

Alice: $d_A, Q_A = d_A \cdot G \xrightarrow{-Q_A}$
 $\xleftarrow{Q_B}$ Bob: $d_B, Q_B = d_B \cdot G$
 segredo = $d_A \cdot Q_B = d_A \cdot d_B \cdot G = d_B \cdot Q_A$

Usa-se só a coordenada x , passada por um KDF.

ECDHE (E = ephemeral): chaves novas a cada sessão → **forward secrecy** (vazar a chave de longo prazo depois não decifra o tráfego passado). É o padrão do TLS 1.3. **X25519** é a variante mais usada.

7.6 ECDSA

Já está no §6.4. Só lembrando o que mais cai: **nonce k único e imprevisível ou a chave privada vaza.**

8. Aprofundamentos (contas na mão)

8.1 Teorema Chinês do Resto — exemplo completo

Problema clássico: achar x com

$$x \equiv 2 \pmod{3}$$

$$x \equiv 3 \pmod{5}$$

$$x \equiv 2 \pmod{7}$$

Receita (construtiva): $N = 3 \cdot 5 \cdot 7 = 105$. Para cada congruência i :

i	n_i	a_i	N_i = N/n_i	y_i = N_i^{-1} mod n_i	a_i · N_i · y_i
1	3	2	35	35 ≡ 2 (mod 3) → 2^{-1} = 2	2 · 35 · 2 = 140
2	5	3	21	21 ≡ 1 (mod 5) → 1^{-1} = 1	3 · 21 · 1 = 63
3	7	2	15	15 ≡ 1 (mod 7) → 1^{-1} = 1	2 · 15 · 1 = 30

$$x = 140 + 63 + 30 = 233, \text{ e } 233 \bmod 105 = 23.$$

Confere: $23 = 3 \cdot 7 + 2 \checkmark$, $23 = 5 \cdot 4 + 3 \checkmark$, $23 = 7 \cdot 3 + 2 \checkmark$.

Por que funciona: cada termo $a_i \cdot N_i \cdot y_i$ vale a_i módulo n_i (porque $N_i \cdot y_i \equiv 1$) e vale **0** módulo todos os outros (porque N_i é múltiplo deles). Ou seja, cada parcela “acende” só a sua congruência. É uma base canônica — a mesma ideia de vetor unitário.

Leitura estrutural: o CRT diz que $Z_{105} \equiv Z_3 \times Z_5 \times Z_7$ como anéis. Trabalhar mod N = trabalhar em paralelo mod cada fator. Guarde isso: é a razão de o RSA-CRT funcionar, e é a razão de o ataque de Håstad funcionar.

8.2 RSA-CRT — como a decifragem fica 4× mais rápida

Custo de exponenciação modular $\approx$ cúbico no tamanho do módulo. Se eu troco uma conta mod N (2048 bits) por **duas** contas mod p e mod q (1024 bits cada), pago $2 \cdot (1/2)^3 = 1/4$. Daí o “4×”.

Pré-computado junto com a chave privada (está lá dentro do seu arquivo `.pem`):

$d_p = d \bmod (p-1)$
 $d_q = d \bmod (q-1)$
 $q_{\text{inv}} = q^{-1} \bmod p$

Decifrar c:

$m_1 = c^{d_p} \bmod p$
 $m_2 = c^{d_q} \bmod q$
 $h = q_{\text{inv}} \cdot (m_1 - m_2) \bmod p$
 $m = m_2 + h \cdot q$

Conferindo com o exemplo miniatura do §5.4 ($p=3$, $q=11$, $N=33$, $e=3$, $d=7$, $c=31$):

$d_p = 7 \bmod 2 = 1$
 $d_q = 7 \bmod 10 = 7$
 $q_{\text{inv}} = 11^{-1} \bmod 3 = 2^{-1} \bmod 3 = 2$

$m_1 = 31^1 \bmod 3 = 1$
 $m_2 = 31^7 \bmod 11 = 9^7 \bmod 11 = 4$
 $h = 2 \cdot (1 - 4) \bmod 3 = -6 \bmod 3 = 0$
 $m = 4 + 0 \cdot 11 = 4 \quad \checkmark \quad (\text{bate com o } m \text{ original})$

! Armadilha real — ataque de Bellcore (fault attack). Se um bit der erro em m_1 (raio cósmico, glitch de tensão, laser num smartcard) e a assinatura errada σ' sair, então $\gcd(\sigma'^e - m, N) = q$. **Um único erro entrega a chave privada.** Por isso toda implementação séria de RSA-CRT **verifica o resultado** ($\sigma^e \stackrel{?}{=} m$) antes de devolver.

8.3 Soma de pontos em curva elíptica — contas de verdade

Curva E: $y^2 = x^3 + 2x + 2$ sobre F_{17} . Ponto base $P = (5, 1)$.

Confere que P está na curva: $5^3 + 2 \cdot 5 + 2 = 137$, e $137 \bmod 17 = 1$. $y^2 = 1^2 = 1$. $\checkmark$

Dobrar: $2P = P + P$

Usa a **tangente**, $\lambda = (3x_1^2 + a) / (2y_1)$:

numerador $= 3 \cdot 5^2 + 2 = 77 \equiv 77 - 68 = 9 \pmod{17}$
denominador $= 2 \cdot 1 = 2$; $2^{-1} \bmod 17 = 9$ (porque $2 \cdot 9 = 18 \equiv 1$)

$$\lambda = 9 \cdot 9 = 81 \equiv 81 - 68 = 13 \pmod{17}$$

$$x_3 = \lambda^2 - 2x_1 = 169 - 10 = 159 \equiv 6 \pmod{17} \quad (17 \cdot 9 = 153)$$

$$y_3 = \lambda(x_1 - x_3) - y_1 = 13 \cdot (5 - 6) - 1 = -14 \equiv 3$$

$$2P = (6, 3). \text{ Confere: } 6^3 + 12 + 2 = 230 \equiv 9 \pmod{17} \text{ e } 3^2 = 9. \checkmark$$

Somar dois pontos diferentes: $3P = 2P + P$

Usa a **reta secante**, $\lambda = (y_2 - y_1)/(x_2 - x_1)$, com $P=(5,1)$, $2P=(6,3)$:

$$\lambda = (3 - 1)/(6 - 5) = 2/1 = 2$$

$$x_3 = \lambda^2 - x_1 - x_2 = 4 - 5 - 6 = -7 \equiv 10 \pmod{17}$$

$$y_3 = \lambda(x_1 - x_3) - y_1 = 2 \cdot (5 - 10) - 1 = -11 \equiv 6$$

$$3P = (10, 6). \text{ Confere: } 10^3 + 20 + 2 = 1022 \equiv 2 \pmod{17} \text{ e } 6^2 = 36 \equiv 2. \checkmark$$

Seguindo assim, o grupo gerado por P tem **19 pontos** ($19P = 0$) — ordem prima, exatamente o que se quer numa curva de crypto.

Três observações que valem a nota da prova:

1. Toda “divisão” virou **inverso modular** (Euclides estendido, §2.2). Inverter é caro — por isso implementações reais usam **coordenadas projetivas/Jacobianas**, que adiam a inversão para o fim.
2. $P + (-P) = 0$: a reta por (x,y) e $(x,-y)$ é **vertical**, não corta a curva em terceiro ponto — daí a necessidade do ponto no infinito.
3. **Double-and-add** é o mesmo algoritmo do §2.3. Para $13P$ (1101_2): $P \rightarrow 2P \rightarrow 3P \rightarrow 6P \rightarrow 12P \rightarrow 13P$. Versão ingênua ramifica conforme o bit e **vaza a chave privada por timing** → use Montgomery ladder (tempo constante).

8.4 Baby-step Giant-step (BSGS)

Resolver $g^x = h$ num grupo de ordem n , sem força bruta.

Truque: escreve $x = i \cdot m + j$, com $m = \lceil \sqrt{n} \rceil$ e $0 \leq i, j < m$. Então $g^{(im+j)} = h \Rightarrow g^j = h \cdot (g^{(-m)})^i$.

1. $m = \lceil \sqrt{n} \rceil$
2. **BABY STEPS:** monta tabela hash $\{ g^j \rightarrow j \}$ para $j = 0..m-1$ ($\sqrt{n}$ memória)
3. $c = g^{(-m)}$ (um inverso, calculado uma vez)
4. **GIANT STEPS:** para $i = 0..m-1$, calcula $y = h \cdot c^i$
se y está na tabela com valor $j \rightarrow x = i \cdot m + j$ ($\sqrt{n}$ tempo)

Exemplo: resolver $5^x \equiv 7 \pmod{23}$. Aqui $g = 5$ é raiz primitiva, ordem $n = 22$, logo $m = \lceil \sqrt{22} \rceil = 5$.

Baby steps — tabela de $5^j \pmod{23}$ para $j = 0..4$:

j	0	1	2	3	4
5^j	1	5	2	10	4

O passo gigante: $c = g^{(-m)} = (5^5)^{-1} = 20^{-1} \equiv 15 \pmod{23}$ (confere: $20 \cdot 15 = 300 = 13 \cdot 23 + 1 \checkmark$)

i	$\gamma = 7 \cdot 15^i \pmod{23}$	está na tabela?
0	7	não
1	$7 \cdot 15 = 105 \equiv 13$	não
2	$13 \cdot 15 = 195 \equiv 11$	não
3	$11 \cdot 15 = 165 \equiv 4$	sim $\rightarrow j = 4$

$x = i \cdot m + j = 3 \cdot 5 + 4 = 19$. Confere: $5^{19} \pmod{23} = 7 \checkmark$

Repare o que aconteceu: em vez de testar 22 expoentes, fiz 5 baby steps + 4 giant steps. Com $n = 10^{12}$, seria 1 milhão em vez de 1 trilhão.

Custo: $O(\sqrt{n})$ tempo e $O(\sqrt{n})$ memória. Para $n \approx 2^{256}$, seriam 2^{128} entradas de tabela — mais memória que átomos acessíveis. Por isso 256 bits basta.

Moral: BSGS é o “raise de baseline”. Qualquer grupo com ordem menor que $\sim 2^{160}$ é quebrável só com isso. É o motivo de existir um **piso** de tamanho de chave.

8.5 Pollard rho

Mesmo custo de tempo do BSGS ($O(\sqrt{n})$) mas com **memória $O(1)$** . É o ataque prático de verdade contra ECDLP.

Ideia — paradoxo do aniversário. Defina um passeio pseudoaleatório no grupo onde cada posição é sempre da forma $g^a \cdot h^b$, com a, b conhecidos:

partição o grupo em 3 regiões S_1, S_2, S_3 por alguma função barata de x

$$\begin{aligned} x \in S_1 &\rightarrow x \leftarrow h \cdot x & (b \leftarrow b+1) \\ x \in S_2 &\rightarrow x \leftarrow x^2 & (a \leftarrow 2a, b \leftarrow 2b) \\ x \in S_3 &\rightarrow x \leftarrow g \cdot x & (a \leftarrow a+1) \end{aligned}$$

Como o grupo é finito, o passeio **entra num ciclo** — o formato ρ que dá nome ao algoritmo (uma “cauda” + um “laço”). Pelo paradoxo do aniversário, isso acontece após $\sim 1.25 \cdot \sqrt{n}$ passos.

Deteção do ciclo (Floyd): roda duas cópias, uma a 1 passo por iteração e outra a 2 passos, até baterem. Memória: dois pontos. Só isso.

Extração da resposta: ao colidir, temos $g^{a_1} h^{b_1} = g^{a_2} h^{b_2}$. Com $h = g^x$:

$$\begin{aligned} a_1 + x \cdot b_1 &\equiv a_2 + x \cdot b_2 \pmod{n} \\ x &\equiv (a_1 - a_2) \cdot (b_2 - b_1)^{-1} \pmod{n} \end{aligned}$$

Uma inversão modular e a chave privada sai.

Pollard rho para fatorar é o primo-irmão: mesmo passeio, mas em $\mathbb{Z}_N$, com $x \leftarrow x^2+1$, procurando $\gcd(|x_i - x_j|, N) > 1$. Acha um fator p em $O(\sqrt{p}) = O(N^{(1/4)})$.

Por que isso fixa os tamanhos de chave: - é **paralelizável** com ganho linear (versão de Van Oorschot–Wiener, com “distinguished points”) — k máquinas dão k vezes mais rápido - é o **melhor ataque conhecido contra ECDLP**, porque não há index calculus lá - logo, curva de n bits $\Rightarrow n/2$ bits de segurança. **256 $\rightarrow$ 128**. Fim.

9. Exercícios (com respostas)

Faça no papel antes de olhar. Respostas logo abaixo de cada bloco.

Bloco A — números e grupos

1. Calcule $\gcd(1071, 462)$ por Euclides.
2. Ache $17^{-1} \bmod 43$ (Euclides estendido).
3. Quanto vale $\varphi(100)$? E $\varphi(143)$?
4. Calcule $7^{222} \bmod 11$ usando Fermat.
5. $\mathbb{Z}_{15}^*$ tem quantos elementos? Liste-os. É cíclico?
6. Em $\mathbb{Z}_{11}^*$, qual a ordem do elemento 3? Ele é gerador?

Respostas A

1. $1071 = 2 \cdot 462 + 147$; $462 = 3 \cdot 147 + 21$; $147 = 7 \cdot 21 + 0 \rightarrow \mathbf{21}$.
2. $43 = 2 \cdot 17 + 9$; $17 = 1 \cdot 9 + 8$; $9 = 1 \cdot 8 + 1$. Voltando: $1 = 9 - 8 = 9 - (17 - 9) = 2 \cdot 9 - 17 = 2(43 - 2 \cdot 17) - 17 = 2 \cdot 43 - 5 \cdot 17 \rightarrow -5 \cdot 17 \equiv 1 \rightarrow \mathbf{17^{-1} = 38}$. (Confere: $17 \cdot 38 = 646 = 15 \cdot 43 + 1 \checkmark$)
3. $100 = 2^2 \cdot 5^2 \rightarrow \varphi = 100 \cdot (1 - \frac{1}{2}) (1 - \frac{1}{5}) = 40$. $143 = 11 \cdot 13 \rightarrow 10 \cdot 12 = 120$.
4. $\varphi(11) = 10$, $222 \bmod 10 = 2 \rightarrow 7^2 = 49 \equiv 5$.
5. $\varphi(15) = 8$: $\{1, 2, 4, 7, 8, 11, 13, 14\}$. **Não é cíclico** — todo elemento tem ordem ≤ 4 ($\mathbb{Z}_{15}^* \cong \mathbb{Z}_2 \times \mathbb{Z}_4$). Só $\mathbb{Z}_n^*$ com $n = 1, 2, 4, p^k, 2p^k$ é cíclico.
6. $3^1=3, 3^2=9, 3^3=5, 3^4=4, 3^5=1 \rightarrow$ **ordem 5**. $|\mathbb{Z}_{11}^*| = 10 \neq 5 \rightarrow$ **não é gerador**.

Bloco B — RSA

7. $p = 5, q = 11, e = 3$. Ache $N, \varphi(N)$ e d .
8. Com essa chave, cifre $m = 9$ e decifre de volta.
9. Alice publica $N = 3233, e = 17$. Ela usou $p = 61, q = 53$. Qual é d ?
10. Por que $e = 65537$ e não $e = 3$ nos padrões modernos?
11. Mostre a maleabilidade: dado $c = m^e$, como produzir a cifra de $2m$ sem saber m ?

Respostas B

7. $N = 55, \varphi = 4 \cdot 10 = 40. d = 3^{-1} \bmod 40 = 27$ ($3 \cdot 27 = 81 = 2 \cdot 40 + 1$).
8. $c = 9^3 = 729 = 13 \cdot 55 + 14 \rightarrow c = 14$. $14^{27} \bmod 55$: $14^2 = 196 \equiv 31$; $14^4 \equiv 31^2 = 961 \equiv 26$; $14^8 \equiv 26^2 = 676 \equiv 16$; $14^{16} \equiv 16^2 = 256 \equiv 36$. $27 = 16 + 8 + 2 + 1 \rightarrow 36 \cdot 16 \cdot 31 \cdot 14 \bmod 55$: $36 \cdot 16 = 576 \equiv 26$; $26 \cdot 31 = 806 \equiv 31$; $31 \cdot 14 = 434 \equiv 9 \checkmark$
9. $\varphi = 60 \cdot 52 = 3120$; $d = 17^{-1} \bmod 3120 = 2753$ ($17 \cdot 2753 = 46801 = 15 \cdot 3120 + 1$). (*É o exemplo da Wikipédia — bom pra treinar.*)
10. $e = 3$ é rápido mas frágil: $m^3 < N$ sem padding quebra direto, e o ataque de broadcast de Håstad precisa de só 3 destinatários. $65537 = 2^{16} + 1$ tem **dois bits 1** (ainda rápido: 17 quadrados + 1 multiplicação) e é grande o bastante para matar esses ataques.
11. $c' = c \cdot 2^e \bmod N$. $\text{Dec}(c') = (m^e \cdot 2^e)^d = (2m)^e \cdot d = 2m$. Nenhum conhecimento de m foi necessário $\rightarrow$ **é por isso que RSA cru não é CCA-seguro**.

Bloco C — DH, ElGamal, assinaturas

12. $p = 23, g = 5$, Alice tem $a = 4$, Bob tem $b = 3$. Qual o segredo comum?

13. No exercício acima, o que Eve vê no fio? Por que ela não consegue o segredo?
14. Em ElGamal, o que acontece se Bob reusa o mesmo r para duas mensagens m_1 e m_2 , e Eve conhece m_1 ?
15. Duas assinaturas ECDSA (r, s_1) e (r, s_2) — mesmo r ! — sobre $e_1 \neq e_2$. Derive k e depois a chave privada d .
16. Por que assinar $H(m)$ em vez de m exige que H seja resistente a colisão, e não só unidirecional?

Respostas C

12. $A = 5^4 = 625 \equiv 4 \pmod{23}$; $B = 5^3 = 125 \equiv 10$. $s = B^a = 10^4 = 10000 \equiv 18$; $s = A^b = 4^3 = 64 \equiv 18$. $\rightarrow 18 \checkmark$
13. Eve vê $p=23$, $g=5$, $A=4$, $B=10$. Precisaria resolver o **log discreto** ($5^a \equiv 4$) ou o **CDH** direto. Com p de 23 é trivial na mão; com p de 3072 bits, não. *A segurança é só o tamanho.*
14. $c_2 = m \cdot h^r$. Duas cifras com o mesmo r dão $c_2^{(1)}/c_2^{(2)} = m_1/m_2$. Sabendo m_1 , sai $m_2 = m_1 \cdot c_2^{(2)}/c_2^{(1)}$. **Nonce reusado = mensagem vazada.** Mesma estrutura do erro do one-time pad reusado.
15. Mesmo $r \Rightarrow$ mesmo k (pois $r = (kG) \cdot x$). De $s = k^{-1}(e + rd)$:

$$s_1 - s_2 = k^{-1}(e_1 - e_2) \rightarrow k = (e_1 - e_2) \cdot (s_1 - s_2)^{-1} \pmod{n}$$

$$d = (s_1 \cdot k - e_1) \cdot r^{-1} \pmod{n}$$

Duas subtrações e duas inversões. **Foi literalmente isso no PS3 e nas carteiras Bitcoin do Android.**
16. Unidirecional só impede achar m a partir de $H(m)$. Mas o forjador não precisa disso: ele pega **duas** mensagens suas m (benigna) e m' (maliciosa) com $H(m) = H(m')$, te faz assinar m , e a **mesma assinatura vale para m'** — Verify só olha o hash. Logo o requisito é **colisão**. É o ataque que aposentou MD5 (certificado falso da CA, Flame, 2012) e SHA-1 (SHattered, 2017).

Bloco D — conceitual (responda em uma frase)

17. Por que $\varphi(pq) = (p-1)(q-1)$ é exatamente a porta dos fundos do RSA?
18. Diferença prática entre CPA-seguro e CCA-seguro — dê um cenário real onde a diferença importa.
19. Por que DDH é falsa em $\mathbb{Z}_p^*$ inteiro, e como o padrão conserta?
20. Explique em uma frase por que ECC de 256 bits $\approx$ RSA de 3072 bits.

Respostas D

17. Quem sabe p, q calcula φ numa multiplicação e daí $d = e^{-1} \pmod{\varphi}$; quem só tem N precisaria fatorar. **Conhecer $\varphi(N)$ é equivalente a fatorar N .**
18. CPA supõe atacante passivo; CCA supõe que ele **manda cifras e observa a resposta**. Cenário: servidor TLS que responde “padding inválido” — Bleichenbacher usa esse único bit para decifrar a chave de sessão. CPA não cobre isso; CCA sim.
19. O **símbolo de Legendre** (é ou não resíduo quadrático) é calculável em tempo polinomial e vaza 1 bit sobre o expoente — suficiente para distinguir g^{ab} de aleatório. Conserto: trabalhar no **subgrupo de ordem prima q** (primo seguro $p = 2q+1$, g gerador dos resíduos quadráticos).
20. Em $\mathbb{Z}_p^*$ existe **index calculus** (subexponencial), então p precisa ser enorme; em curvas só existe Pollard rho genérico ($O(\sqrt{n})$), então n bits $\Rightarrow n/2$ bits de segurança, e $256/2 = 128$.

10. Colinhas finais

Quem resolve o quê

Esquema	Grupo	Hipótese	Serve pra
RSA	$\mathbb{Z}^*_N$	fatoração / RSA	cifrar + assinar
DH / ECDH	$\mathbb{Z}^*_p$ / curva	CDH / DDH	trocar chave
ElGamal	$\mathbb{Z}^*_p$ / curva	DDH	cifrar
DSA / ECDSA	subgrupo ordem q / curva	DL	assinar

Erros clássicos (todos já derrubaram sistemas reais)

1. RSA sem padding
2. Reusar o nonce k no (EC)DSA
3. DH sem autenticação $\rightarrow$ MITM
4. Confundir cifragem com assinatura (“cifrar com a privada” não é assinar)
5. Gerador de aleatórios fraco — mata tudo, não importa a matemática
6. Verificar padding v1.5 por parsing em vez de re-codificar
7. Comparação de tempo não constante $\rightarrow$ side-channel

O que usar hoje

- **Troca de chave:** X25519 (ECDHE)
- **Assinatura:** Ed25519, ou ECDSA P-256
- **Cifragem pública:** HPKE / ECIES; RSA-OAEP se obrigado
- **Hash:** SHA-256 ou SHA-3
- **Simétrico:** AES-256-GCM ou ChaCha20-Poly1305

Para responder em aula, saiba dizer em uma frase

- por que $\varphi(pq) = (p-1)(q-1)$ é o segredo do RSA
- por que RSA cru não é CPA-seguro
- por que DH sozinho sofre MITM
- por que ECC usa chaves menores (ausência de index calculus)
- por que reusar k no ECDSA vaza a chave privada
- diferença CPA $\times$ CCA, CDH $\times$ DDH, MAC $\times$ assinatura

Referências

- Menezes, van Oorschot, Vanstone — *Handbook of Applied Cryptography* (grátis: cacr.uwaterloo.ca/hac) — cap. 2 (números), 3 (hipóteses), 4 (primalidade), 8 (cifragem pública), 11 (assinaturas)
- Katz & Lindell — *Introduction to Modern Cryptography* — melhor para as definições de segurança (CPA/CCA/EUF-CMA)
- Hoffstein, Pipher, Silverman — *An Introduction to Mathematical Cryptography* — melhor para curvas elípticas
- Nigel Smart — *Cryptography Made Simple*